Sets: HashSet, LinkedHashSet and TreeSet

7 min read · Updated 28 Jul 2026 · Topic 20 of 65

Bookmark · Mark done

Why it matters

Sets are the collection people under-use. A great deal of code does `if (!list.contains(x)) list.add(x)` — which is O(n) per check and turns a loop into an O(n²) one — when a `Set` would be O(1) and say what it means. Interviewers also like sets because "how does `HashSet` guarantee uniqueness?" has an answer that instantly reveals whether you understand the previous topics.

The concept, simply

A `Set` holds no duplicates, where "duplicate" means `equals` returns true. The three implementations mirror the three maps exactly, because that is literally what they are: **`HashSet` is a `HashMap` whose values are all a single dummy object.**

JAVA

Copy

```java
// The actual implementation in the JDK, lightly paraphrased:
public class HashSet<E> implements Set<E> {
    private transient HashMap<E, Object> map;
    private static final Object PRESENT = new Object();   // one shared du

    public boolean add(E e) {
        return map.put(e, PRESENT) == null;   // put returns the OLD value
    }                                         // null means it was not the
    public boolean contains(Object o) { return map.containsKey(o); }
}
```

behaviour. That single sentence answers most set questions.

How it works

	Backed by	add/contains	Order	Nulls
`HashSet`	`HashMap`	O(1) average	None	One null
`LinkedHashSet`	`LinkedHashMap`	O(1) average	Insertion	One null
`TreeSet`	`TreeMap`	O(log n)	Sorted	**None**

The one genuinely distinct behaviour is `add`'s return value: it returns `true` if the element was added and `false` if it was already present. That makes it a *test-and-insert in one atomic-looking call*, which is the idiomatic way to detect duplicates in a single pass.

Code examples

The pattern that replaces most duplicate-detection code:

JAVA

Copy

```java
// ✗ O(n²) — contains() scans the whole list every iteration.
List<String> seen = new ArrayList<>();
for (String s : input) if (!seen.contains(s)) seen.add(s);

// ✓ O(n), and add() tells you whether it was new.
Set<String> seen = new HashSet<>();
for (String s : input) {
    if (!seen.add(s)) System.out.println("duplicate: " + s);
}

// Deduplicate while KEEPING the original order — LinkedHashSet, one line:
List<String> deduped = new ArrayList<>(new LinkedHashSet<>(input));
```

Set algebra, which is far more readable than the loops people write instead:

JAVA Copy

```java
Set<String> a = new HashSet<>(Set.of("x", "y", "z"));
Set<String> b = Set.of("y", "z", "w");

Set<String> union = new HashSet<>(a);
union.addAll(b);                        // [x, y, z, w]

Set<String> intersection = new HashSet<>(a);
intersection.retainAll(b);              // [y, z]

Set<String> difference = new HashSet<>(a);
difference.removeAll(b);                // [x]

// ⚠ All three MUTATE the receiver — always copy first, as above. And
// removeAll has a performance cliff: if the argument is a List rather tha
// a Set, it does a linear scan per element, making it O(n × m).
```

`TreeSet`'s navigation, mirroring `TreeMap`:

JAVA Copy

```java
TreeSet<Integer> s = new TreeSet<>(List.of(10, 20, 30, 40));
s.first();          // 10
s.last();           // 40
s.floor(25);        // 20  — greatest ≤ 25
s.ceiling(25);      // 30  — smallest ≥ 25
s.headSet(30);      // [10, 20]
s.subSet(15, 35);   // [20, 30]
s.descendingSet();  // [40, 30, 20, 10]

// And the same compareTo-defines-equality rule as TreeMap:
Set<String> ci = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
ci.add("Hello"); ci.add("HELLO");
System.out.println(ci.size());    // 1
```

The mutable-element trap, which is the same bug as a mutable map key:

JAVA

Copy

```
Set<List<String>> set = new HashSet<>();
List<String> list = new ArrayList<>(List.of("a"));
set.add(list);
System.out.println(set.contains(list));   // true

list.add("b");                            // mutate AFTER insertion —
                                          // List.hashCode() depends on co
System.out.println(set.contains(list));   // false ✗ — even for the same c
System.out.println(set.size());           // 1 — it is in there, unreachab
```

Under the hood

Why the dummy-value design? Reusing `HashMap` means `HashSet` inherits every optimisation the map ever receives — treeification, the improved resize, the spread function — with no duplicated code. The cost is one wasted reference per element, which is a good trade. It also means the JDK maintains one hard data structure instead of two, which is why `HashSet` has essentially never had a bug of its own.

`Set.of` **is deliberately hostile to mistakes** in two ways worth knowing. It *throws* on duplicate arguments at construction time rather than silently collapsing them — `Set.of("a", "a")` is an `IllegalArgumentException`, which catches copy-paste errors. And its iteration order is deliberately *randomised per JVM run*, specifically so that code cannot accidentally come to depend on it. That randomisation surprises people who see different output across runs and assume something is broken.

The concurrent set is a small gap in the API that comes up: there is no `ConcurrentHashSet` class. The answer is `ConcurrentHashMap.newKeySet()`, which returns a set backed by a concurrent map — the same dummy-value trick, applied to the concurrent implementation. Knowing that method name is a small but real signal.

Common mistakes

- `list.contains` **in a loop.** O(n²). Use a `Set`.

- **Elements without `equals` / `hashCode`.** A `HashSet` of such objects deduplicates nothing — every instance is unique.
- **Mutating an element after adding it.** Same bug as a mutable map key; the element becomes unreachable.
- **Expecting `HashSet` to preserve order.** Use `LinkedHashSet`.
- **Forgetting `addAll` / `retainAll` / `removeAll` mutate the receiver.** Copy first.
- **`removeAll` with a `List` argument.** Degrades to O(n × m). Pass a `Set`.
- **Duplicates in `Set.of( ... )`.** Throws at construction, by design.
- **Null in a `TreeSet`.** Throws — nothing to compare.

Interview lens

Q: "How does `HashSet` guarantee uniqueness?"
A: It does not implement anything itself — it wraps a `HashMap`, storing each element as a key with a single shared dummy object as the value. Uniqueness comes entirely from the map's key uniqueness, which means it comes from `hashCode` and `equals`. `add` returns `map.put(e, PRESENT) == null`, which is why it tells you whether the element was new.

Q: "`HashSet` vs `TreeSet` vs `LinkedHashSet`?"
A: They mirror the maps exactly. `HashSet` is O(1) with no order; `LinkedHashSet` is O(1) with insertion order for two extra references per element; `TreeSet` is O(log n), sorted, no nulls, and adds navigation methods like `floor`, `ceiling` and `subSet`.

Q: "What happens if you add an object without `hashCode` to a `HashSet`?"
A: Every instance gets a distinct identity hash, so no two objects are ever considered duplicates — the set silently accepts logical duplicates. Adding the same conceptual value twice gives you a set of size 2, which is exactly the `equals` / `hashCode` contract failing in its most visible form.

A: `new ArrayList<>(new LinkedHashSet<>(list))`, or `list.stream().distinct().toList()`. Both are O(n). A plain `HashSet` would deduplicate but scramble the order.

Q: "Is there a `ConcurrentHashSet`?"

A: No class by that name. Use `ConcurrentHashMap.newKeySet()`, which gives a set backed by a `ConcurrentHashMap` — the same dummy-value approach applied to the concurrent map. `Collections.newSetFromMap` is the general form if you need it over some other map.

Q: "Why does `Set.of("a", "a")` throw?"

A: Deliberate design. The immutable factories reject duplicate arguments with `IllegalArgumentException` rather than silently collapsing them, because a duplicate in a literal is almost always a typo. Their iteration order is also randomised per JVM run, specifically so nobody can come to depend on it.

Cheat-card

- `HashSet` **IS a** `HashMap` with a shared dummy value. Everything about map keys applies.
- `add()` returns **false if already present** — test-and-insert in one call.
- `HashSet` O(1) · `LinkedHashSet` O(1) + insertion order · `TreeSet` O(log n) + sorted, no nulls.
- Dedupe keeping order: `new ArrayList<>(new LinkedHashSet<>(list))`.
- Set algebra: `addAll` / `retainAll` / `removeAll` — they **mutate the receiver**; pass Sets, not Lists.
- Concurrent set → `ConcurrentHashMap.newKeySet()`.
- `Set.of` throws on duplicates; its order is **randomised per JVM run**.
- Never mutate an element after adding it.

1. How does HashSet guarantee uniqueness?

- ◯ It uses reference identity only
- ◯ It scans the set on each add to check for duplicates
- ◯ It wraps a HashMap, storing each element as a KEY with a single shared dummy value — uniqueness comes entirely from map key uniqueness, i.e. from hashCode and equals
- ◯ It sorts elements and compares neighbours

Check answer

2. How do you deduplicate a list while preserving the original order?

- ◯ Collections.sort followed by a manual scan
- ◯ new ArrayList<>(new HashSet<>(list))
- ◯ list.stream().sorted().distinct().toList()
- ◯ new ArrayList<>(new LinkedHashSet<>(list)), or list.stream().distinct().toList()

Check answer

3. There is no ConcurrentHashSet class. What do you use instead?

- ◯ ConcurrentHashMap.newKeySet()
- ◯ CopyOnWriteArraySet for all cases
- ◯ A ConcurrentLinkedQueue with manual duplicate checks

Check answer

Previous
LinkedHashMap and TreeMap

Next
ConcurrentHashMap and Concurrent Collections

LEARN MORE

ARTICLES & DOCS

- HashSet — API docs
  docs.oracle.com
- TreeSet — API docs
  docs.oracle.com

VIDEOS



Internal Working and implementation of hashmap and hashset | Java Interview Questions | Code Decode
Code Decode

Hand-picked references for this lesson.